

计算机系统基础实 验报告

实验二：Bomblab



学生姓名：俞浩坤

学 号：22300240034

日 期：2023.10.26

拆弹成功的截图：

二、关卡推演过程：

2、phase_2: 函数首先将输入存储到-0x28(%rbp)中，接着将 phase_2_nums 数组首地址赋给%rsi，之后将输入传回%rdi 后调用<read six numbers>函数，由函数

名可猜测其读取了六个数字，由于在调用前传入了数组首地址，可能将读入的六个数存储在了数组中，在接下来的过程中可对此进行验证。

之后函数将数组的首地址赋给-0x10(%rbp)，将 1 赋值给-0x4(%rbp)，再将-0x4(%rbp)的值与 5 进行比较，如果大于 5 则跳到<phase_2+139>退出函数，注意到<phase_2+133>行对-0x4(%rbp)进行了加一，之后又跳回<phase_2+53>将其与 5 进行比较，因而-0x4(%rbp)可以用来表示循环进行的次数，且循环最多进行 6 次，现将循环次数记作 i，开始分析循环过程，首先将数组中第 8 个元素赋值给%edx，用 x/gx 指令查得其值为 0x00000001ffffffc，由于 edx 只存储 32 位因而值为-2。

```
(gdb) x/gx 0x40637c
0x40637c <phase_2_nums+28>: 0x00000001ffffffc
```

然后把 i 赋值给%eax，通过左移 2 位使其值为 4i 再将其减 4 赋给%rcx，接着把数组首元素的值地址赋给%rax，由 x/gx 可查得其值为 0x0000000400000002。

```
(gdb) x/gx 0x406360
0x406360 <phase_2_nums>: 0x0000000400000002
```

再把其加上%rcx 的值传递给%eax，其存储值为数组首地址+4i-4，即数组中第 i 个元素，将其乘以-2 再加上数组中第九个元素后传入-0x14(%rbp)中，由 x/gx 可查得数组中第九个元素值为 1，因而存入的值为 1-4nums[i-1]。

```
(gdb) x/gx 0x406380
0x406380 <phase_2_nums+32>: 0x0000000000000001
```

接着再把 i+4 的传给%rdx，再把数组首地址传给%rax，将两者相加后所得地址的值赋给%eax，可得%eax 中存储的值为 nums[i]，接着将%eax 与-0x14(%rbp)值相比较，为使炸弹不爆炸要使两者相等即 1-4nums[i-1] = nums[i]，由于 nums[0] = 2，因而之后的 6 个元素由递推式可求得为-7, 29, -115, 461, -1843, 7373，这即是我们需要输入的 6 个值且存储在了 nums 数组的第 2 到 7 个元素中，符合之前对于函数功能的猜测，由此得到第二关的密码为：-7 29 -115 461 -1843 7373。

输入密码，通过第二关，再一次验证对于函数功能的猜测。

```
Phase 1 defused. How about the next one?
-7 29 -115 461 -1843 7373
That's number 2. Keep going!
```

3、phase_3：函数首先将输入的三个参数存入-0x11(%rbp)，-0x10(%rbp)和-0xc(%rbp)，根据地址顺序输入的三个参数按顺序存放在-0xc(%rbp)，-0x10(%rbp)和-0x11(%rbp)，通过 x/s 查看存入%rsi 的地址可知输入的三个参数为两个整数和一个字符。

```
(gdb) x/s 0x403248
0x403248: "%d %d %c"
```

之后将输入传回%rdi，将%eax 值赋为 0，调用<__isoc99_sscanf@plt>函数，该函数为统计输入参数的函数，将其返回值与 3 相比较，如果不相等则引爆炸弹，即只要我们输入的参数个数为 3 则不会引爆，接着将-0xc(%rbp)值与 610 相比较，如果相等则跳到<phase_3+334>，跳转后接着将 119 赋值给-0x1(%rbp)，将 987 与-0x10(%rbp)相比较，如果相等则跳转到<phase_3+383>，跳转后将-0x1(%rbp)即 119 与-0x11(%rbp)相比较，如果相等则不会引爆炸弹离开函数，由此可得使输入的 3 个数值为 610, 987, 119 则不会引爆炸弹，由于第三个输入为字符因而为 w，因而第三关的一个密码为 610 987 w，本题根据不同的跳转可得到不同的密码，

但我们只需得出其中一组即可，输入密码，通过第三关。

```
That's number 2. Keep going!  
610 987 w  
Halfway there!
```

4、phase_4: 函数首先将输入的参数存入-0x18(%rbp)，通过 x/s 查看存入%rsi 的地址可知输入的参数为长整形。

```
(gdb) x/s 0x4032e0  
0x4032e0:      "%lld"
```

之后将输入传回%rdi，将%eax 值赋为 0，调用<__isoc99_sscanf@plt>函数，该函数为统计输入参数的函数，将其返回值与 1 相比较，如果不相等则将%al 设为 1，后让其与自身相与，如果相与后结果不为 0 则引爆炸弹，因而不引爆输入参数个数应为 1，接着将输入右移 32 位后的后 32 位即原来的 33-64 位存入-0x4(%rbp)，将输入的后 32 位存入-0x4(%rbp)，首先判断后 32 位是否小于等于 0 或者大于 14，如果有一个条件满足则使%edx 为 1，接着判断前 32 位是否小于等于 0 或者大于 14，如果有一个条件满足则使%eax 为 1，再将%edx 和%eax 相或后再使其与自身相与，如果结果不为 0 则引爆炸弹，因而不引爆输入参数要使输入的前 32 位和后 32 位均大于 0 小于 14，接着将输入的前 32 位赋值给%edi，调用<_ZL4hopei>函数，通过 disas 0x4017ac 可查看该函数的汇编代码。

```
(gdb) disas 0x4017ac  
Dump of assembler code for function _ZL4hopei:  
0x00000000004017ac <+0>:      endbr64  
0x00000000004017b0 <+4>:      push    %rbp  
0x00000000004017b1 <+5>:      mov     %rsp,%rbp  
0x00000000004017b4 <+8>:      sub     $0x20,%rsp  
0x00000000004017b8 <+12>:     mov     %edi,-0x14(%rbp)  
0x00000000004017bb <+15>:     cmpl    $0x0,-0x14(%rbp)  
0x00000000004017bf <+19>:     jne     0x4017c8 <_ZL4hopei+28>  
0x00000000004017c1 <+21>:     mov     $0x1,%eax  
0x00000000004017c6 <+26>:     jmp     0x4017f2 <_ZL4hopei+70>  
0x00000000004017c8 <+28>:     mov     -0x14(%rbp),%eax  
0x00000000004017cb <+31>:     sar     %eax  
0x00000000004017cd <+33>:     mov     %eax,%edi  
0x00000000004017cf <+35>:     call    0x4017ac <_ZL4hopei>  
0x00000000004017d4 <+40>:     mov     %eax,-0x4(%rbp)  
0x00000000004017d7 <+43>:     mov     -0x14(%rbp),%eax  
0x00000000004017da <+46>:     and     $0x1,%eax  
0x00000000004017dd <+49>:     test    %eax,%eax  
0x00000000004017df <+51>:     je      0x4017ec <_ZL4hopei+64>  
0x00000000004017e1 <+53>:     mov     -0x4(%rbp),%eax  
0x00000000004017e4 <+56>:     imul    %eax,%eax  
0x00000000004017e7 <+59>:     shl     $0x2,%eax  
0x00000000004017ea <+62>:     jmp     0x4017f2 <_ZL4hopei+70>  
0x00000000004017ec <+64>:     mov     -0x4(%rbp),%eax  
0x00000000004017ef <+67>:     imul    %eax,%eax  
0x00000000004017f2 <+70>:     leave   %eax  
0x00000000004017f3 <+71>:     ret
```

该函数为递归调用函数，每次以输入右移 1 位作为新参数输入，若参数为 0 则返回 1，若参数为奇数则返回参数的平方，否则返回参数的平方乘以 4，注意到在调用完函数后会将函数返回值与 0x1000000 比较，如果不相等则将%al 设置为 1，并让其与自身相与，如果结果不为 0 则引爆炸弹，因而不引爆输入参数要使函数返回值为 0x1000000，由于递归调用到最后 1 层结果一定为 1，1 的平方仍为 1，因而倒数第二层调用结果一定为 4，又因为 0x1000000 为 4 的 12 次方，如果在接下来的

三层中先返回结果的平方乘以 4 再返回两次结果的平方则正好为 4 的 12 次方，因而函数输入应为 1100，即 10 进制的 12，所以输入的高 32 位 12，低 32 位为 1-14 中的任意一位数，取低 32 位为 1，由于 64 位二进制数超过了计算器的转换上线，因而采用百度的 16 进制转换器得到一个密码 51539607553。

二进制 八进制 十进制 16进制 32进制 更多: 16进制

c00000001

转换到: 二进制 八进制 十进制 16进制 32进制 更多: 10进制 转换

16进制转换为10进制的结果:

51539607553

输入密码，通过第四关。

```
Halfway there!
51539607553
So you got that one. Try this one.
```

5、phase_5: 函数首先将输入的两个参数存入-0x34(%rbp), -0x38(%rbp)，通过 x/s 查看存入%rsi 的地址可知输入的参数为一个字符串和一个整数。

```
(gdb) x/s 0x4032e5
0x4032e5: "%s %d"
```

接着和前几关一样调用<__isoc99_sscanf@plt>函数判断输入的参数个数是否为 2，如果不为 2 则引爆炸弹，因而输入的参数要为 2，接着将输入的的第一个参数与存储在%rsi 的字符串相比较，调用 <strcmp@plt>函数使如果相同则使返回值为 0，不相同则使返回值为 1，接着将结果与自身相与如果不为 0 则跳转<phase_5+114>，猜测本关和第三关一样存在多个通关密码，由于选取最后一组密码能使阅读的汇编代码最少，因而选择直接跳转，跳转后执行同样操作发现仍旧不是最后一组密码再次跳转到<phase_5+164>，再次执行相同操作，发现这次如果仍不相同则会爆炸因而通过 x/s 查看存入%rsi 的地址得知第一个参数的一个答案为：冲冲冲~，接着将 0x10 赋值给%edi，调用<_Znwm@plt>函数，通过 disas 0x401180 查看函数汇编代码。

```
(gdb) disas 0x401180
Dump of assembler code for function _Znwm@plt:
0x0000000000401180 <+0>:      endbr64
0x0000000000401184 <+4>:      bnd jmp *0x4eb5(%rip)      # 0x406040 <_Znwm@got.plt>
0x000000000040118b <+11>:     nopl 0x0(%rax,%rax,1)
End of assembler dump.
(gdb) disas 0x406040
Dump of assembler code for function _Znwm@got.plt:
0x0000000000406040 <+0>:      adcb $0x40, (%rax)
0x0000000000406043 <+3>:      add %al, (%rax)
0x0000000000406045 <+5>:      add %al, (%rax)
0x0000000000406047 <+7>:      add %dl, 0x4010(%rax)
```

发现该函数似乎只是分配了空间并没有对寄存器的值做出修改，因而可能和通过密码没有关系，所以先跳过该函数，如果不能通过后面的函数找到密码再回来看此函数，之后将%rax 的值 0 赋给%rdi 后调用<_ZN10worldline3C2Ev>函数，通过 disas 0x401fc8 查看函数汇编代码。

```
(gdb) disas 0x401fc8
Dump of assembler code for function _ZN10worldline3C2Ev:
0x0000000000401fc8 <+0>:    endbr64
0x0000000000401fcc <+4>:    push    %rbp
0x0000000000401fcd <+5>:    mov     %rsp,%rbp
0x0000000000401fd0 <+8>:    sub     $0x10,%rsp
0x0000000000401fd4 <+12>:   mov     %rdi,-0x8(%rbp)
0x0000000000401fd8 <+16>:   mov     -0x8(%rbp),%rax
0x0000000000401fdc <+20>:   mov     %rax,%rdi
0x0000000000401fdf <+23>:   call    0x401e88 <_ZN9worldlineC2Ev>
0x0000000000401fe4 <+28>:   lea     0x3d35(%rip),%rdx    # 0x405d20 <_ZTV10worldline3+16>
0x0000000000401feb <+35>:   mov     -0x8(%rbp),%rax
0x0000000000401fef <+39>:   mov     %rdx,(%rax)
0x0000000000401ff2 <+42>:   mov     -0x8(%rbp),%rax
0x0000000000401ff6 <+46>:   movq    $0x1124fd,0x8(%rax)
0x0000000000401ffe <+54>:   nop
0x0000000000401fff <+55>:   leave
0x0000000000402000 <+56>:   ret
End of assembler dump.
(gdb) disas 0x401e88
Dump of assembler code for function _ZN9worldlineC2Ev:
0x0000000000401e88 <+0>:    endbr64
0x0000000000401e8c <+4>:    push    %rbp
0x0000000000401e8d <+5>:    mov     %rsp,%rbp
0x0000000000401e90 <+8>:    mov     %rdi,-0x8(%rbp)
0x0000000000401e94 <+12>:   lea     0x3efd(%rip),%rdx    # 0x405d98 <_ZTV9worldline+16>
0x0000000000401e9b <+19>:   mov     -0x8(%rbp),%rax
0x0000000000401e9f <+23>:   mov     %rdx,(%rax)
0x0000000000401ea2 <+26>:   nop
0x0000000000401ea3 <+27>:   pop     %rbp
0x0000000000401ea4 <+28>:   ret
End of assembler dump.
(gdb)
```

该函数首先通过调用 <_ZN9worldlineC2Ev>函数进行初始化,然后将 0x3d35(%rip)的地址即 0x405d20 赋值给%rdx,再将其赋给%rax 取地址的值,之后把 0x1124fd 赋值给%rax 加 8 的偏移量的位置,之后函数结束,再将%rax 取地址的值赋给%rax 再把其加上 0x10 后变为 0x405d30,以第二个参数作为输入调用以该地址所存的值为地址的函数,首先用 x/gx 查看该地址值为 0x40202a。

```
(gdb) x/gx 0x405d30
0x405d30 <_ZTV10worldline3+32>: 0x000000000040202a
```

再通过 disas 0x40202a 查看该函数汇编代码。

```
(gdb) disas 0x40202a
Dump of assembler code for function _ZN10worldline35dmailEi:
0x000000000040202a <+0>:    endbr64
0x000000000040202e <+4>:    push    %rbp
0x000000000040202f <+5>:    mov     %rsp,%rbp
0x0000000000402032 <+8>:    mov     %rdi,-0x8(%rbp)
0x0000000000402036 <+12>:   mov     %esi,-0xc(%rbp)
0x0000000000402039 <+15>:   cmpl    $0x7e7,-0xc(%rbp)
0x0000000000402040 <+22>:   sete    %al
0x0000000000402043 <+25>:   movzbl  %al,%eax
0x0000000000402046 <+28>:   pop     %rbp
0x0000000000402047 <+29>:   ret
```

函数将第二个参数作为输入,并将其与 0x7e7 进行比较,如果相等则返回 1,不相等则返回 0,接着退出函数,将返回值与自身相与,如果结果为 0 则跳转到 <phase_5+267>将%eax 赋值为 1 然后跳转到<phase_5+279>使其与自身相与得到 1 然后引爆炸弹,为使炸弹不爆炸应当使函数返回为 1 即使第二个参数等于 0x7e7 即 2023,由于至此密码两个参数均寻找完毕,密码为冲冲冲~ 2023,因而不需要继续看剩余的函数,和第三关一样本关根据三个字符串可能对应三组正确密码,

但只需找到一组即可，输入密码，通过第五关。

```
So you got that one. Try this one.  
冲冲冲~ 2023  
Good work! On to the next...
```

6、phase_6: 和第二题一样函数在调用<read_six_numbers>函数后读入了 6 个数，并将其存进了 <phase_6_nums>数组里，接着将数组首地址赋值给-0x10(%rbp)，将循环次数存入-0x4(%rbp)，现将循环次数记为 i，函数将%eax 赋值为 i，再将其乘以 4 赋值给%rdx，即%rdx 值为 4i，再将其加上数组首地址后取地址的值存储在%eax 中，即每次循环所存值为数组中第 i 个元素，接着判断该元素，若该元素大于 6 或者与自身相与的结果小于等于 0 则会引爆炸弹，因而输入的 6 个数都要大于 0 小于等于 6，之后跳出循环，将数组首地址作为输入后调用<build_target>函数，通过 `disas 0x401a96` 查看该函数的汇编代码。

```
(gdb) disas 0x401a96  
Dump of assembler code for function build_target:  
0x0000000000401a96 <+0>:      endbr64  
0x0000000000401a9a <+4>:      push    %rbp  
0x0000000000401a9b <+5>:      mov     %rsp,%rbp  
0x0000000000401a9e <+8>:      push    %r15  
0x0000000000401aa0 <+10>:     push    %r14  
0x0000000000401aa2 <+12>:     push    %r13  
0x0000000000401aa4 <+14>:     push    %r12  
0x0000000000401aa6 <+16>:     push    %rbx
```

发现该函数过于复杂，一开始尝试分析但把自己绕晕了，后来发现该函数再最后调用了一个<check_answer>的函数，通过 `disas 0x401a2b` 查看其汇编代码，发现其在每次核对答案时进行了一次判断，如果不符合判断条件则结束函数。

```
0x0000000000401a75 <+74>:     mov     (%rax),%eax  
0x0000000000401a77 <+76>:     cmp     %eax,%edx  
0x0000000000401a79 <+78>:     jge     0x401a89 <check_answer+94>
```

因而想到可以用 gdb 在跳转前添加断点的方式来核实密码每一位是否正确，如果正确在 continue 之后则会进行下一次判断，如果不正确则会直接跳出函数，注意到如果不正确函数返回值最终为 0，将返回值与 1 做异或后所得结果为 1，再将其与自身相与结果仍为 1，之后会引爆炸弹，因而如果输入的当前位数不对则会立刻爆炸，考虑到每一位只有 6 种情况，且猜测每位数均不相同，因而最多只要尝试 21 次就可得出答案，所以采用 gdb 来得到答案，首先通过 `b *0x401a79` 添加断点。

```
(gdb) b *0x401a79  
Breakpoint 1 at 0x401a79
```

接着输入答案开始尝试，注意到只有在输入的参数为 6 且均大于 0 小于等于 6 时才会进入该函数否则炸弹在之前就会爆炸，因而采用将当前尝试的位数和之后的位数均用同一个值的方法，从小到大开始尝试，首先尝试 1 1 1 1 1 1，发现在输入两次 c 后才爆炸，说明答案第一位就是 1。

```

Good work! On to the next...
1 1 1 1 1 1

Breakpoint 1, 0x0000000000401a79 in check_answer ()
(gdb) c
Continuing.

Breakpoint 1, 0x0000000000401a79 in check_answer ()
(gdb) c
Continuing.
BOOM!!!

```

接着输入 1 2 2 2 2 2，发现仍然是输入两次 c 后爆炸，说明第二位答案不是 2。

```

Good work! On to the next...
1 2 2 2 2 2

Breakpoint 1, 0x0000000000401a79 in check_answer ()
(gdb) c
Continuing.

Breakpoint 1, 0x0000000000401a79 in check_answer ()
(gdb) c
Continuing.
BOOM!!!

```

通过反复尝试 14 次后得到密码 1 6 4 3 5 2。输入密码，通过第六关，同时表明本方法可行。

```

Good work! On to the next...
1 6 4 3 5 2
Cool! your skill on Reverse Engineering is great.

```

7、secret_phase: 根据提示，其解锁条件在 main.cpp 中有所给出。

请注意，Secret的解锁条件在第三关中进行了提示，或许你能在 `main.cpp` 的某个函数中发现一些违和之处.....

在 main.cpp 中发现如果输入 secret_key 则会进入 secret_phase 且在最初定义其为 0。

```

/* Note to self: The secret key should be zero when release! */
if (secret_key) {
    /* The code shouldn't go here */

    slow_put("Welcome to the secret phase of Bomb+!\n");
    slow_put("It is called");
    slow_slow_put("... THE STEINS;GATE\n");

    read_line(input);
    secret_phase(input);

    puts("You are really a Master of Reverse Engineering!");
    true_ending();
}

```



```
char input[40];
long secret_key = 0;
```

又由于解锁条件在第三关中给出了提示，首先尝试在第三关密码后多输入 0 看是否会给出提示。

```
You have 6 phases with which to blow yourself up. Have a nice day!
Computer science is not a boring subject
Phase 1 defused. How about the next one?
-7 29 -115 461 -1843 7373
That's number 2. Keep going!
610 987 w 0
Halfway there!
51539607553
So you got that one. Try this one.
冲冲冲~ 2023
Good work! On to the next...
1 6 4 3 5 2
Cool! your skill on Reverse Engineering is great.
[Inferior 1 (process 637) exited normally]
```

发现炸弹正常结束，并没有提示，但同时也没有爆炸，没有爆炸或许就是提示，此时猜测是不是要在所有密码后都输入 0 才会解锁。

```
You have 6 phases with which to blow yourself up. Have a nice day!
Computer science is not a boring subject 0
Phase 1 defused. How about the next one?
-7 29 -115 461 -1843 7373 0
That's number 2. Keep going!
610 987 w 0
Halfway there!
51539607553 0
So you got that one. Try this one.
冲冲冲~ 2023 0
Good work! On to the next...
1 6 4 3 5 2 0
Cool! your skill on Reverse Engineering is great.
Welcome to the secret phase of Bomb+!
It is called... THE STEINS;GATE
```

输入后发现真的解锁了 secret_phase，使用 `disas secret_phase` 查看汇编代码。

```
0x0000000000401ddf <+42>: mov    -0x18(%rbp),%rax
0x0000000000401de3 <+46>: lea    0x152f(%rip),%rsi          # 0x403319
0x0000000000401dea <+53>: mov    %rax,%rdi
0x0000000000401ded <+56>: mov    $0x0,%eax
0x0000000000401df2 <+61>: call   0x401160 <__isoc99_sscanf@plt>
0x0000000000401df7 <+66>: cmp    $0x1,%eax
0x0000000000401dfa <+69>: setne  %al
0x0000000000401dfd <+72>: test   %al,%al
0x0000000000401dff <+74>: je     0x401e06 <secret_phase+81>
0x0000000000401e01 <+76>: call   0x402048 <explode_bomb>
0x0000000000401e06 <+81>: mov    -0x8(%rbp),%eax
Type <RET> for more, q to quit, c to continue without paging--RET
0x0000000000401e09 <+84>: xor    -0x4(%rbp),%eax
0x0000000000401e0c <+87>: mov    %eax,-0x8(%rbp)
0x0000000000401e0f <+90>: mov    -0x8(%rbp),%eax
0x0000000000401e12 <+93>: cmp    $0xbaadf00d,%eax
0x0000000000401e17 <+98>: je     0x401e1e <secret_phase+105>
0x0000000000401e19 <+100>: call   0x402048 <explode_bomb>
0x0000000000401e1e <+105>: nop
0x0000000000401e1f <+106>: leave
```

函数首先将输入传入-0x18(%rbp)，接着通过 x/s 查看存入%rsi 的地址可知输入的参数为一个无符号整数。

```
(gdb) x/s 0x403319
0x403319:      "%ud"
```

接着依旧是调用<__isoc99_sscanf@plt>函数判断输入的参数个数是否为 1，如果不为 1 则引爆炸弹，因而输入的参数要为 1，接着函数将存储在-0x8(%rbp)的值和存储在-0x4(%rbp)的值进行异或运算，将结果与 0xbaadf00d 进行比较，如果不相等则引爆炸弹，因而要使结果与 0xbaadf00d 相等，由于-0x8(%rbp)和-0x4(%rbp)在之前的代码并没有出现过，因而只能用 gdb 来查看两者的值，首先来查看-0x8(%rbp)的值，注意到在 0x401e06 处将其赋值给%eax，通过 b * 0x401e09 添加断点后使用 i r 来查看%eax 的值，之后查看-0x4(%rbp)的值，同样通过 b * 0x401e0c 添加断点后使用 i r 来查看异或后的值，首先输入 1 来进行观察。

```
1
test

Breakpoint 1, 0x0000000000401e09 in secret_phase ()
(gdb) i r
rax          0x1          1
rbx          0x0          0
```

```
Breakpoint 2, 0x0000000000401e0c in secret_phase ()
(gdb) i r
rax          0xdeadcdcf   3735929055
rbx          0x0          0
```

发现-0x8(%rbp)中存储的值为输入的 1，将-0x4(%rbp)的值与 1 异或后的值为 0xdeadcdcf，猜测-0x8(%rbp)储存的是输入，-0x4(%rbp)储存的值为常量，接着再输入 0 来进行观察。

```
Breakpoint 1, 0x0000000000401e09 in secret_phase ()
(gdb) i r
rax          0x0          0
rbx          0x0          0
rcx          0x18         24
rdx          0x0          0
```

```
Breakpoint 2, 0x0000000000401e0c in secret_phase ()
(gdb) i r
rax          0xdeadcdde   3735929054
rbx          0x0          0
rcx          0x18         24
rdx          0x0          0
```

发现-0x8(%rbp)储存的仍是输入，-0x4(%rbp)储存的值仍为常量，因而可验证猜想，由于任何事与 0 异或后的结果仍为其自身，因而-0x4(%rbp)储存的值为 0xdeadcdde，要使炸弹不爆炸，即使输入等于 0xdeadcdde 与 0xbaadf00d 异或后的结果，即 1677734099。输入密码，通过秘密关卡，至此炸弹拆除完毕。

```
Welcome to the secret phase of Bomb+!  
It is called... THE STEINS;GATE  
1677734099  
test
```

You are really a Master of Reverse Engineering!

[illegible]

You have enter the true ending and defeat JWC's Bomb+!
In this worldline, JWC'As is not limited to 30%!
You are the true hero!